

Unità di apprendimento 2

**I socket e la
comunicazione con
i protocolli TCP/UDP**



The background of the slide is a blue gradient with a pattern of binary code (0s and 1s) in a lighter blue color. On the left side, there is a partial view of a laptop screen and keyboard.

Unità di apprendimento 2

Lezione 2

**La connessione tramite
socket**

Generalità

- Il concetto di **socket** è stato sviluppato come estensione diretta del **paradigma UNIX di I/O su file**, che si basa sulla sequenza di operazioni **open-read-write-close**, dove:
 - **open** permette di accedere a un file
 - **read/write** consentono di accedere ai contenuti del file
 - **close** termina l'utilizzo del file
- Le socket a questa struttura aggiungono l'insieme dei parametri necessari a realizzare la connessione tra macchine remote, ovvero:
 - gli **indirizzi**
 - il **tipo di protocollo**
 - il **protocollo** e il numero di **porta**

Generalità

- Ogni sistema operativo fornisce, tramite le proprie API, un insieme di **meccanismi** che permettono ai programmi di utilizzare e mettere in comunicazione **protocolli diversi** senza che il programmatore debba gestire direttamente i dettagli di basso livello
 - ad es. HTTP => TCP => IP => Ethernet
- le **socket API** sono delle specifiche API che hanno origine con Berkeley BSD UNIX e che oggi sono disponibili in Windows, Solaris e Linux

Generalità

- Di seguito le funzioni presenti sia in C che in Java (rif. Java Doc Socket e Java Doc ServerSocket):
 - **socket()/server socket()**: crea un nuovo socket
 - **close()**: termina l'utilizzo di un socket (client e server)
 - **bind()**: collega un indirizzo di rete a un socket (client e server)
 - **listen()**: attende messaggi in ingresso (server)
 - **accept()**: accetta una connessione in ingresso (server)
 - **connect()**: crea una connessione con un host remoto (client)
 - **send()/write()**: trasmette dati su una connessione attiva (client e server)
 - **recv()/read()**: riceve dati da una connessione attiva (client e server)
- Su python sono analoghe (rif. Python Doc)

Famiglie e tipi di socket

- Esistono varie **famiglie di socket** (domini)
 - Ogni famiglia riunisce i socket che utilizzano gli **stessi protocolli sottostanti** (Protocol Family)
 - Ciascuna supporta un sottoinsieme di stili di comunicazione e possiede un proprio **formato di indirizzamento** (Address Family)
- Tra le famiglie di socket ricordiamo:
 - **Internet socket (AF_INET)**, che permette il trasferimento di dati tra processi posti su macchine **remote** connesse tramite una LAN o Internet
 - **Unix Domain socket (AF_UNIX)**, che permette il trasferimento di dati tra processi sulla **stessa** macchina Unix

Famiglie e tipi di socket

- La principale differenza tra i socket nei domini AF_UNIX e AF_INET è l'**indirizzo**:
 - in AF_UNIX è praticamente il **pathname** valido nel file system della macchina
 - in AF_INET è specificato da due valori:
 - **indirizzo IP** (32 bit)
 - **numero di porta** (16 bit)
- I socket AF_INET sono fondamentalmente di **tre tipi** e per ciascuno abbiamo una diversa modalità di connessione:
 - **stream socket**
 - **datagram socket**
 - **raw socket**, utilizzati nello sviluppo di protocolli

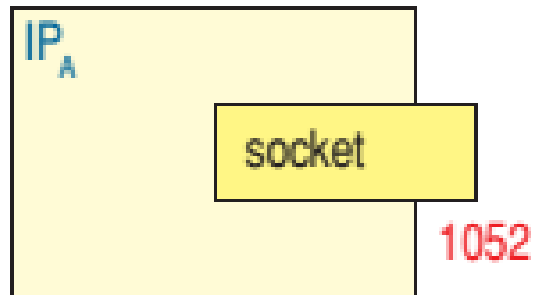
Stream socket

- Con gli **stream socket** (**SOCK_STREAM**) si realizza una connessione (handshake) **sequenziale** tipicamente **asimmetrica, affidabile e full-duplex**
- Ogni processo crea il proprio **endpoint** (il socket), poi:
 - il server si mette in ascolto, in attesa di un collegamento, e quando gli arriva una richiesta la esaudisce mediante la primitiva **accept()** che **crea un nuovo socket dedicato alla connessione** (stessa porta, ma nuova association)
 - il client si pone in coda sul socket del server con la primitiva **connect()** e crea **implicitamente** il binding con la porta locale (solitamente non si invoca `bind()` in modo esplicito) e solo in questo momento il client associa alla sua socket una porta locale (quest'ultima gli serve solo quando effettivamente comunica a differenza del server!) e attende di essere “accettato” dal server

Stream socket

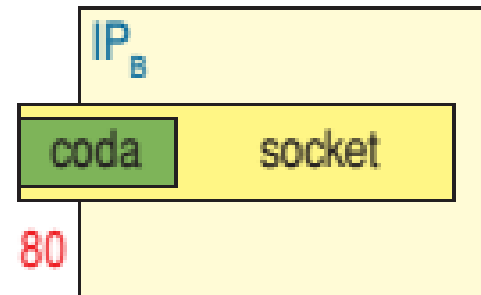
- Vediamo graficamente i singoli passaggi che realizzano la connessione tra server e client
- A Inizializzazione dei processi.

Client



Crea un **socket**

Server

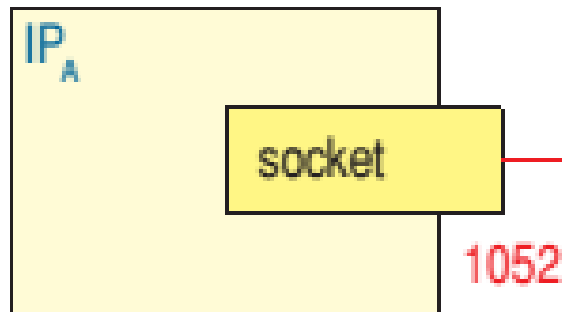


Crea un **socket**
Lo connette alla porta 80
Si pone in ascolto

Stream socket

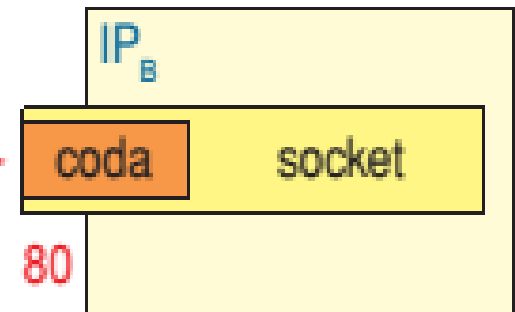
- ⑤ Il **client** effettua la richiesta di collegamento.

Client



Richiesta
di collegamento

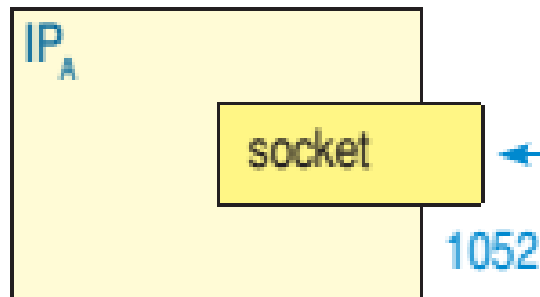
Server



Stream socket

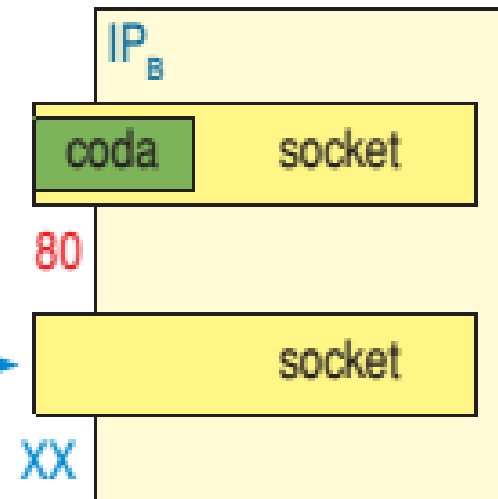
- © Il **server** accetta la richiesta e stabilisce il collegamento.

Client



Canale virtuale

Server

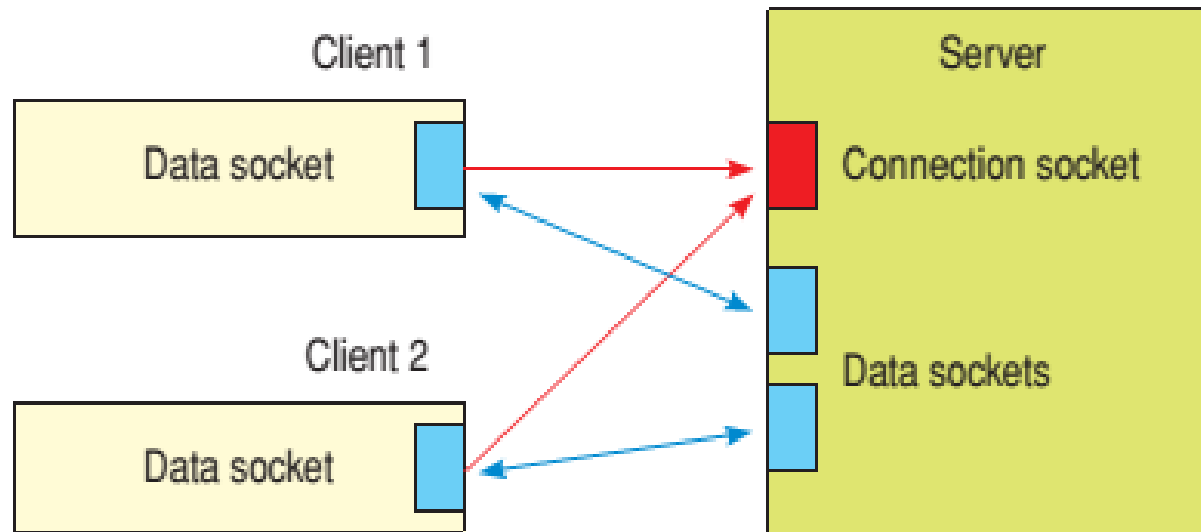


Stream socket

- Il server accetta la richiesta e realizza “un canale virtuale” tra il client e un **nuovo** socket del server (indicato XX)
- In tale modo lascia il primo libero per le **ulteriori** richieste da parte di altri client
- I due processi si scambiano i dati (tramite **read()** e **write()**) fino alla chiusura del canale, che viene effettuata rispettivamente mediante la chiamata della primitiva **close()**
- Il protocollo TCP è basato su questo tipo di socket che realizza il modello “uno a uno”!

Stream socket

- Dunque in un server TCP abbiamo due tipi di socket:
 - un tipo per accettare connessioni (condiviso), che chiamiamo **connection socket** (in Java ServerSocket)
 - un tipo per le operazioni send() e receive(), non condiviso con altri client, che chiamiamo **data socket** (in Java Socket)



Datagram socket

- Con i **datagram socket** (**SOCK_DGRAM**) viene realizzata la comunicazione che permette di scambiare dati senza connessione (niente handshake) ovvero senza garantire ordine o arrivo dei pacchetti, quindi si ha una comunicazione inaffidabile
- Permettono quindi di inviare da un socket a **più** destinazioni e ricevere su un socket da **più** sorgenti
- In generale realizzano quindi il modello “molti a molti” e sono supportati nel dominio Internet dal protocollo UDP

Trasmissione multicast

- TCP è sempre unicast, mentre UDP è principalmente utilizzato per trasmissioni multicast
- Nella comunicazione di tipo multicast
 - un insieme di processi formano un **gruppo di multicast**
 - un messaggio spedito da un processo a quel gruppo viene recapitato a **tutti gli altri partecipanti** appartenenti al gruppo
- Tipiche applicazioni multicast sono:
 - videoconferenze
 - massive multiplayer games
 - DNS
 - chat, instant messaging, applicazioni P2P, ecc.

Trasmissione multicast

- Per implementare un sistema multicast è necessario
 - definire uno **schema di indirizzamento** dei gruppi
 - definire un supporto che registri la **corrispondenza** tra un gruppo e i partecipanti
 - ottimizzare l'**uso della rete** nel caso di invio di pacchetti a un gruppo (tramite multicast router)
- Il protocollo più utilizzato per il multicast in Internet è l'**IGMP** (**Internet Group Management Protocol**)
- IGMP serve a garantire la trasmissione dei messaggi relativi alla **costituzione dei gruppi** tra host e multicast router a essi direttamente collegati

Trasmissione multicast

- Le **API multicast** devono quindi contenere primitive per:
 - **unirsi** a un gruppo di multicast (**join**)
 - **lasciare** un gruppo di multicast (**leave**)
 - **spedire** messaggi a un gruppo
 - **ricevere** messaggi indirizzati a un gruppo del quale l'host fa parte
- La gestione dei gruppi è di tipo dinamico:
 - un host può unirsi o abbandonare un gruppo in qualsiasi momento e può appartenere **contemporaneamente** a più gruppi
 - non è necessario appartenere a un gruppo per poter **inviare** a esso dei messaggi (non è ovviamente vero il contrario!)
 - i membri del gruppo possono appartenere alla **medesima** rete o a reti fisiche **differenti**
- Vengono utilizzati indirizzi IP di classe D

Trasmissione multicast

- Gli indirizzi possono essere:
 - **permanenti**, l'indirizzo di multicast viene assegnato dalla IANA (Internet Assigned Numbers Authority) e rimane assegnato a quel gruppo anche se in un certo momento **non** ci sono più partecipanti connessi
 - **temporanei**, richiedono la definizione di un opportuno protocollo per **evitare conflitti** nell'attribuzione degli indirizzi ai gruppi ed esistono solo fino al momento in cui esiste **almeno un** partecipante
- La comunicazione multicast utilizza il paradigma connectionless

Nei router viene mantenuta la corrispondenza tra l'indirizzo di multicast e gli indirizzi IP dei singoli host che partecipano al gruppo